字符串入门

DMU-XCPC 集训队

Nil

Dalian Maritime University – Aleatory Acceptism of Encounter

January 23, 2026

字符串匹配

字符串匹配问题

定义：

- 又称模式匹配，简单来讲就是对于主串 S，能不能在其内部找到字串 T 的问题，其中 T 也可以称作模式串。

朴素的做法肯定是一下一下比较，两个指针来表示匹配过程的写法.

```
std::vector<int> match(char *s, char *t, int n, int m) {
  std::vector<int> ans;
  int i, j;
  for (i = 0; i < n - m + 1; i++) {
    for (j = 0; j < m; j++) {
      if (s[i + j] != t[j]) break;
    }
    if (j == m) ans.push_back(i);
  }
  return ans;
}
```

不难看出，BF 算法如果在最坏的时间复杂度下是 $O(nm)$ 的，这显然不符合我们的预期，于是在这里，我们引入我们今天要学习的第一个算法。

字符串哈希

因为我们知道，当我们逐字符比较的时候，每一次都要比每个字符，我们想到，如果使用“字符串的哈希值”呢？这里的哈希值指的是按照某种规则将字符串转而成的数值。

那么如何计算哈希值呢？

我先来举一个例子，如果我们把这个字符串里的每一个字符的 ASCII 码相加，作为其哈希值怎么样？

> 显然不行？没考虑顺序？或是什么之类的。

字符串哈希

这样的现象就叫做「哈希冲突」！

用一个形象的例子来说，你有一个名字，但是名字重复的概率很大；而指纹，身份证号等物件的重复概率就小得多。

因为我们的计算机内存是有限的，所以我们并不能设计出完全不可能冲突的哈希，这里给出一个算法竞赛中通常使用的哈希计算方式：

哈希的计算

$$（哈希值）= B^{K-1}T[1] + ... + B^1T[K-1] + B^0T[K]$$

B - 可以理解为 B 进制，也可以理解为多项式的变量
$T[i]$ - 字符串 T 中的第 i 个字符

字符串哈希

然而，随着字符串变长，哈希值会变成一个巨大的数字。例如 $B = 100$ 的时候，仅仅 100 个字符的字符串就会产生长达 200 位的天文数字。因此，通常会对一个素数 M 取模作为最终的哈希值。尽管原本这套算法可以保证，对于不同的字符串，一定有不同的哈希值，但是取模后不一定，但由于哈希值原则上呈随机分布，不同字符串哈希值相同的概率约为 $\frac{1}{M}$，可能性极低。因此只要 M 足够大就不会有问题。

但有些问题中，即使 M 达到数万级甚至亿级仍然可能不足。这种情况下，准备多个素数 M，当所有哈希值都一定时才判定「字符串相等」的方法会更可靠。此外，若存在恶意设计的测试样例或可能根据程序特征生成恶意输入时，使用随机化的 B, M 数值等策略可能会更有效。

我们学会了哈希的计算方法，但是我们如何高效的算出呢？
对于字符串 $S[l:r]$，直接计算需要 $O(r-l)$ 时间，因此我们可以考虑预处理！
令 H_i 为 $S[1:i]$ 的哈希值（1-indexed）。
不难看出，对于 $S[1:i]$ 来说，只是在 $S[i:i-1]$ 后添加了 $T[i]$，所以 $H_i = B \times H_{i-1} + T[i]$
那么 $S[l:r]$ 的哈希值就可以通过 $H_r - B^{r-l+1} \times H_{l-1}$ 快速计算了！
在这里 B^{r-l+1} 的数值只需要通过快速幂求，或预处理出即可，因为一定有 $r - l + 1 \leq N$。

补充：关于证明，为什么这样可行？

最后，让我们证明一下为什么 $S[l:r]$ 的哈希值为 $H_r - B^{r-l+1} \times H_{l-1}$ ？
首先根据哈希值的定义，H_r 的数值是：

$$B^{r-1}T[1] + ... + B^{r-l} + 1T[l-1] + B^{r-l}T[l] + ... + B^0T[r]$$

另一方面，$H_r - B^{r-l+1} \times H_{l-1}$ 的数值是：

$$B^{r-1}T[1] + ... + B^{r-l} + 1T[l-1]$$

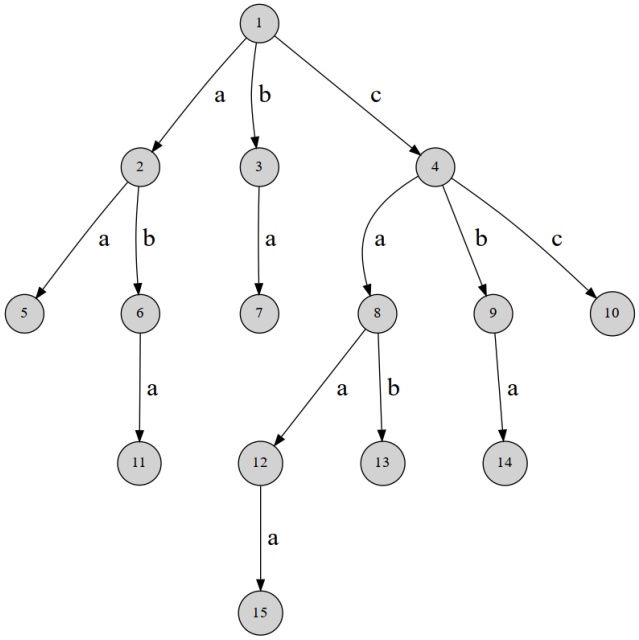
因此，第一个式子减去第二个式子的值后，剩下的部分与根据定义计算的 $S[l:r]$ 的哈希值时，得到的答案相同。

https://www.luogu.com.cn/problem/P2957

字典树（TRIE）

定义：

- 也称前缀树，顾名思义，就是一个像字典一样的树。



Trie

利用字符串的公共前缀来减少查询时间，最大限度地减少无谓的字符串比较.

字典树（TRIE）

现在，我们需要考虑一个字典树需要包含什么：

- 节点
- 插入字符串操作
- 查找字符串操作

https://www.luogu.com.cn/problem/P2580

KMP 算法

我们先来看看 KMP 算法的想法，既然 BF 匹配每一次都要从头，而有没有一种写法可以更快的实现跳转呢？

这里我们要引入一个概念：(0-indexed)

- 前缀函数 $\pi[i]$：字符串 $s[0:i]$ 的「最长相等真前缀与真后缀」的长度。
- 真前缀：前缀 $\mathrm{Prefix}(S,i) = S[0:i]$，而真前缀就是除了 S 本身的 S 的前缀。
- 真后缀：后缀 $\mathrm{Suffix}(S,i) = S[i:|S|-1]$，而真后缀就是除了 S 本身的 S 的后缀。

举个例子：

KMP 算法

对于字符串「abcabcd」

$\pi[0] = 0\ (\mathrm{a})$

$\pi[1] = 0\ (\mathrm{ab})$

$\pi[2] = 0\ (\mathrm{abc})$

$\pi[3] = 1\ (\mathrm{abca}，\text{前缀a} = \text{后缀a})$

$\pi[4] = 2\ (\mathrm{abcab}，\mathrm{ab} = \mathrm{ab})$

$\pi[5] = 3\ (\mathrm{abcabc}，\mathrm{abc} = \mathrm{abc})$

$\pi[6] = 0\ (\mathrm{abcabcd})$

KMP 算法

KMP 算法：利用前缀函数在 O(n+m) 时间内完成字符串匹配。

核心思想：匹配失败时，不回退文本指针，利用前缀函数跳过已匹配部分。

具体是怎么实现的呢？

$$\begin{aligned} &\text{ababcabcabababd} \\ &\text{ababd} \quad \text{在 d 的位置失败} \\ \pi[i]: &\ 00120 \quad \pi[3] = 2 \\ &\quad\ \text{ababd} \quad \text{从第二位开始比} \end{aligned}$$

好的，那么我们如何计算它呢？

朴素的想法需要枚举长度，枚举开始点，再比较字符串，显然是个 $O(|S|^3)$ 的算法。

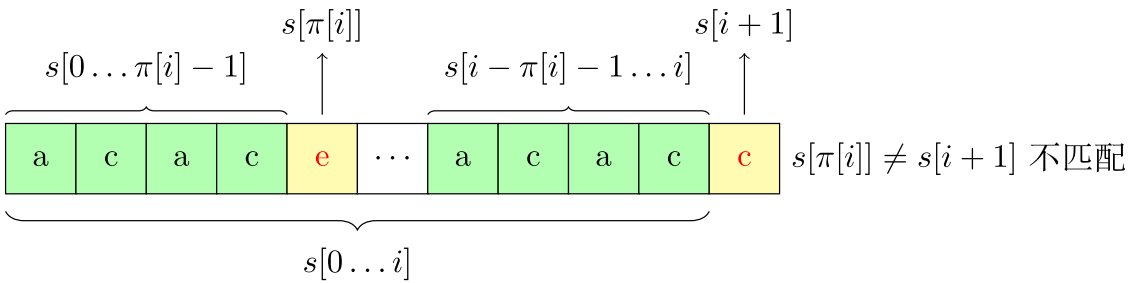
如何优化？

第一个观察：我们发现，相邻的前缀函数值至多加 1。

那么我们知道，对于 $\pi[i+1]$，必然要求其新增的 $s[i+1]$ 也与对应的字符匹配，也就是 $s[i+1] = s[\pi[i]]$，这个时候有 $\pi[i+1] = \pi[i] + 1$.

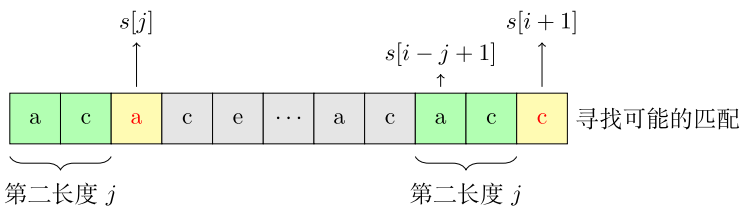
KMP 算法

这个时候，当移动到下一个位置的时候，要么维持不变，要么减少，要么增加 1.
这个时候的复杂度就变成了 $O(|S|^2)$ 的了.
还能继续优化么？
显然可以，这个时候我们考虑，当 $s[i+1] \neq s[\pi[i]]$ 的时候，怎么办？



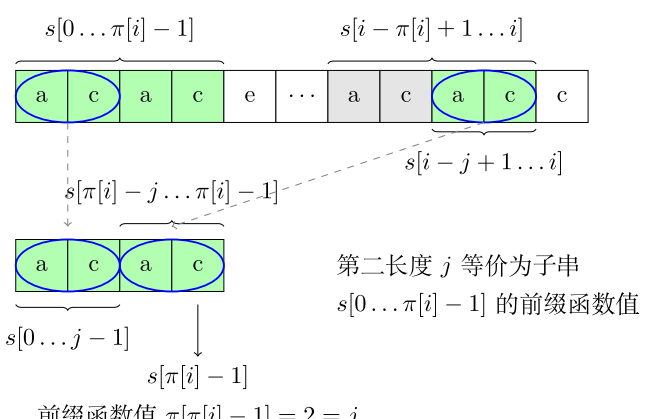
我们希望找到对于 $s[0:i]$，仅次于 $\pi[i]$ 的第二长度 j，使得 $s[0:j-1] = s[i-j+1:i]$
如果我们找到了这个 j，那么只需要比较 $s[i+1]$ 和 $s[j]$，如果相等，就有 $\pi[i+1] = j+1$，反之我们就要找到第二个仅次于 j 的值 $j^{(2)}$，如此反复.

KMP 算法



观察发现，由于有 $s[0:\pi[i]-1]=s[i-\pi[i]+1:i]$，对于第二长度 j，有性质：

$$s[0:j-1]=s[i-j+1:i]=s[\pi[i]-j:\pi[i]-1]$$



KMP 算法

所以我们终于找到了「藏在藏宝图后的宝藏」!
就是下面这个递推式子!

$$j^{(n)} = \pi\left[j^{(n-1)} - 1\right],\ \left(j^{(n-1)} > 0\right)$$

于是我们得到了一个 $O(|S|)$ 的写法!